SALAZAR — Viva Exam Quick Review Guide

UML Structural Associations, Package/Component Diagrams & System Requirements

1. UML Class & Object Associations

Class Diagram Relationships

- COMPOSITION (▲ / ■) **Strong "owns/contains" lifecycle connection** — child objects cannot exist without parent.
  - **User ▲ Conversation:** User owns conversations.
  - **Conversation ▲ Message:** Conversation contains messages.
  - **User ▲ Document:** User owns uploaded documents.
  - **Document ▲ Chunk:** Documents are divided into processed chunks.
  - **Chunk ▲ Embedding:** Chunk is represented by a vector embedding.
  - **Task ▲ Workflow:** Task creates and owns a multi-agent workflow.
  - **Workflow ▲ AgentExecution:** Workflow contains ordered execution steps.
- AGGREGATION (♢) **Weak "has-a" connection** — child objects can exist independently of parent.
  - **VectorStore ♢ Embedding:** Indexes embeddings.
  - **Agent ♢ Tool:** Agents have access to executable tools.
- ASSOCIATION (⟶) **Structural relationship or interaction** between independent entities.
  - **User ⟶ Task:** User creates tasks.
  - **Agent ⟶ AgentExecution:** Agent performs execution steps.
  - **Task ⟶ Agent:** Task requires specialized agents.
  - **Agent ⟶ Model:** Agent uses a local LLM model instance.
  - **ResourceManager ⟶ Model:** ResourceManager manages model states.
  - **Model ⟶ ResourceAllocation:** Model receives allocated CPU/RAM/VRAM resources.
  - **ResourceManager ⟶ CPU / RAM / GPU:** Monitors hardware resources.
  - **Orchestrator ⟶ Workflow:** Executes workflows.
  - **Conversation ⟶ Task:** Conversation generates tasks.
  - **Message ⟶ Model / Agent:** Messages are generated by specific models/agents.
- DEPENDENCY (⇢) **Runtime operational dependencies** (<<use>>).
  - **TaskRouter ⇢ Task:** Analyzes task types.
  - **TaskRouter ⇢ Agent:** Selects appropriate agents.
  - **Agent ⇢ RAG:** Agent uses RAG for external context retrieval.
  - **Task ⇢ RAG:** Task may require context retrieval.

Object Diagram Links (Concrete Snapshot Instances)

Runtime Trace: `u1:User` owns `conv1:Conversation` (contains `msg1`, `msg2`) ➔ generates `task1:Task` ➔ `router1:TaskRouter` analyzes task & selects `agent1:Agent` ➔ `task1` creates `wf1:Workflow` (contains `exec1:AgentExecution` performed by `agent1`) ➔ `agent1` uses `model1:Model` managed by `rm1:ResourceManager` (tracks `alloc1:ResourceAllocation`) ➔ `msg2` generated by `model1`.

2. UML Package Dependencies

Subsystem dependency chain (Layered / Decoupled Architecture):

Source Package	Target Package	Dependency Type & Description
User Interface	**API Gateway**	`<<use>>` React/Next.js UI calls FastAPI endpoints
API Gateway	**Orchestration**	`<<use>>` FastAPI forwards requests to Orchestrator, TaskRouter & Workflows
Orchestration	**Agent System & Model Mgmt**	`<<use>>` Orchestrator coordinates agents & Resource Manager
Agent System	**Model Mgmt & Knowledge/ RAG**	`<<use>>` Agents execute prompts via local LLM & pull RAG context
Knowledge/ RAG	**Persistence**	`<<use>>` RAG stores/searches embeddings in Vector DB & SQLite
UI / Orch / Agent	**Persistence**	`<<use>>` Systems persist history, configurations & tasks

3. UML Component Provided & Required Interfaces

- INTERFACE **Frontend ⟷ Backend API:** `REST / WebSocket API`
- INTERFACE **Backend API ⟷ Orchestrator:** `Orchestration API`
- INTERFACE **Orchestrator ⟷ Task Router / Agent Manager / Resource Manager:** `Routing API`, `Agent Exec API`, `Resource API`
- INTERFACE **Agent Manager ⟷ Model Runtime & RAG Engine:** `Inference API`, `Retrieval API`
- INTERFACE **RAG Engine ⟷ Embedding Service & Vector Database:** `Embedding API`, `Vector Query API`
- INTERFACE **Resource Manager ⟷ Host Hardware:** `Hardware API` (monitors CPU/RAM/GPU)
- INTERFACE **Components ⟷ Relational DB:** `Persistence API` (SQLite)

4. System Requirements

Functional Components

- CHAT **Local AI Chat:** Accepts natural-language prompts, streams tokens, renders Markdown/code blocks, and persists message/agent/model history.
- ROUTING **Intelligent Model Routing:** Classifies task complexity & type, automatically selects suitable agents/models (with manual override for power users).
- WORKFLOW **Multi-Agent Workflows:** Coordinates sequential or concurrent execution across specialized agents (Planner, Coder, Reviewer), passing intermediate outputs across stages.
- RAG **Document Processing & RAG:** Ingests PDFs, chunks text, generates embeddings, stores in Vector DB, runs similarity search, and injects context into LLM.
- CODING **Coding Assistance:** Handles code generation, debugging, refactoring, and directory-restricted file access.
- RESOURCE **Resource & Lifecycle Management:** Detects host CPU/RAM/VRAM, manages model states (Load ➔ Run ➔ Idle ➔ Unload), and prevents memory overflow.

Non-Functional Components

- PERF **Performance:** Prefers 3B/4B models for fast latency; caps parallel workloads at max 2 LLM executions on reference hardware (16GB RAM, 6GB VRAM).
- SEC **Security & Privacy:** 100% offline with zero external telemetry. Restricts file system agents strictly to user-authorized project directories.
- REL **Reliability & Availability:** Isolates model errors to prevent app crashes, recovers gracefully from ingestion failures, and preserves DB integrity on power loss.
- PORT **Portability:** Executes directly from removable media (USB/external SSD) on Windows using relative paths.
- MAINT **Maintainability:** Decoupled modular architecture allows updating individual models, agents, or tools independently.